

TP1 - SIMULATION DES 5 SPOF QuickCommerce - Architecture Monolithique

Durée : 2h30

Format : Binôme ou Trinôme (2-3 étudiants)

Prérequis : Aucun (tout sera installé pendant le TP)

Objectif : Installer QuickCommerce et simuler les 5 Single Points of Failure

NB: Nos réponses seront écrites en bleus et la donnée en noir

OBJECTIFS PÉDAGOGIQUES

À la fin de ce TP, vous saurez :

-  Installer Docker Desktop
-  Cloner un projet depuis Git
-  Démarrer une application avec Docker Compose
-  **Identifier les 5 SPOF** par la pratique
-  **Simuler des pannes** et mesurer leurs impacts
-  **Calculer les pertes financières** d'un downtime
-  Proposer des solutions d'amélioration

CONTEXTE BUSINESS

QuickCommerce :

- E-commerce BtoC
- CA : 12M€/an = 1M€/mois = 33k€/jour
- CA horaire : **5,000€/h = 83€/minute**
- 50,000 visiteurs/jour
- Hébergement : Serveur unique chez prestataire historique

Incident Black Friday 2024 :

- 4h de downtime total
- 200,000€ de perte
- 5 SPOF identifiés

Votre mission : Reproduire ces pannes en laboratoire pour comprendre les risques.

PARTIE 1 : INSTALLATION (45 min)

Exercice 1.1 : Installation Docker Desktop (15 min)

Chaque étudiant sur sa machine

Étape 1 : Télécharger Docker Desktop

 **Lien** : <https://www.docker.com/products/docker-desktop/>

Choisir :

- Windows : Docker Desktop for Windows
- Mac : Docker Desktop for Mac
- Linux : Docker Engine

Étape 2 : Installer Docker Desktop

Windows :

1. Double-cliquer sur **Docker Desktop Installer.exe**
2. Suivre l'assistant (paramètres par défaut)
3. **Redémarrer** le PC si demandé
4. Lancer Docker Desktop

Attendre : Logo de la baleine en bas à droite = "Docker is running"

Étape 3 : Vérifier l'installation

Ouvrir PowerShell

docker --version

Résultat attendu : Docker version 24.x.x

docker-compose --version

Résultat attendu : Docker Compose version v2.x.x

Tester

docker run hello-world

Doit afficher : "Hello from Docker!"

✅ **Checkpoint** : Docker fonctionne

📄 **À REMPLIR** :

Étudiant	Docker installé	Version	Statut
Martin BONNAND	✅ Oui ☐ Non	_____	✅ OK
Teddy SMAILI	✅ Oui ☐ Non	_____	✅ OK
Axel WOEL LALA	✅ Oui ☐ Non	29.2.1	✅ OK

Tatiana ABI SALEH ☐ Oui ❌ Non

❌

❌

Problème rencontré par Tatiana : Son ordinateur étant celui prêté par l'ISEP, elle n'a pas accès au BIOS et ne peut donc pas activer **Intel Virtualization Technology** (or **AMD-V**),

- Pendant ce temps, elle va travailler et aider sur le pc d'Axel :) comme le ferait une femme

Exercice 1.2 : Installation Git (10 min)

Vérifier si Git est installé

git --version

Si installé : ✅ Passer à l'exercice 1.3

Si pas installé :

Étape 1 : Télécharger Git

🌐 **Lien** : <https://git-scm.com/download/win>

OU installer via winget :

winget install --id Git.Git -e --source winget

Étape 2 : Vérifier

git --version

Résultat attendu : git version 2.x.x

✅ Git installé

Exercice 1.3 : Cloner le Projet QuickCommerce (10 min)

Étape 1 : Créer un dossier de travail

Créer le dossier

```
mkdir C:\TP-Architecture
```

```
cd C:\TP-Architecture
```

Étape 2 : Cloner le repository

Cloner depuis Git (URL fournie par l'enseignant)

```
git clone https://github.com/tsantafamenontsoa/quickcommerce-monolithe.git
```

Aller dans le dossier

```
cd quickcommerce-monolithe
```

Étape 3 : Vérifier la structure

Lister les fichiers

```
ls
```

Résultat attendu :

```
# code/
```

```
# docker-compose.yml
```

```
# README.md
```

✅ Projet cloné

Exercice 1.4 : Démarrer QuickCommerce (10 min)

Chaque étudiant sur sa machine

Étape 1 : Aller dans le dossier du projet

```
cd C:\TP-Architecture\quickcommerce-monolithe\archi-docker-compose
```

Étape 2 : Lire le docker-compose.yml

Voir le contenu

```
cat docker-compose.yml
```

OU ouvrir avec VSCode/Notepad

```
code docker-compose.yml
```

Observer :

- Combien de services ?
 - Il y a 2 services
 - Quels sont-ils ?
 - Les 2 services sont quickcommerce_app et quickcommerce_mysql
 - Port de l'API ?
 - Le port de l'API est 3000
 - Port MySQL ?
 - Le port de MySQL est 3306
-

Étape 3 : Build et démarrer

```
docker-compose -f docker-compose-production.yml up --build -d
```

Attendre : 1-2 minutes (téléchargement images + build)

Logs affichés :

[+] Building 45.2s (12/12) FINISHED

[+] Running 3/3

✓ Network quickcommerce-network Created

✓ Container quickcommerce_mysql Started

✓ Container quickcommerce_app Started

✓ Application démarrée

Étape 4 : Vérifier les conteneurs

`docker-compose ps`

OU

`docker ps`

Résultat attendu :

NAME	STATUS	PORTS
quickcommerce_app	Up (healthy)	0.0.0.0:3000->3000/tcp
quickcommerce_mysql	Up (healthy)	0.0.0.0:3306->3306/tcp

Si STATUS = "Exited" : Problème ! Voir les logs :

`docker-compose logs`

Étape 5 : Tester l'API

Health check

`curl http://localhost:3000/health`

Résultat attendu :

{"status":"OK"}

Produits

`curl http://localhost:3000/api/products`

Résultat attendu : JSON avec 10 produits

✅ QuickCommerce fonctionne !

 **À REMPLIR**

Étudiant	Conteneurs UP	API répond	Produits affichés
Teddy Smaili	Oui	Oui	Oui

Étudiant	Conteneurs UP	API répond	Produits affichés
Axel WOEL LALA	YES	YES	YES
Tatiana ABI SALEH	Non	Non	Non
Martin BONNAND	Oui	Oui	Oui

Statut OK (fonctionne) :

```
smailliteddy@MacBook-Air-de-Smaili archi-docker-compose % docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED
STATUS        PORTS                                NAMES
5bfc52260226   quickcommerce-app:1.0               "docker-entrypoint.s..." 16 minutes ago
Up 16 minutes (healthy)   0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp quickcommerce_app
25293e7ef701   mysql:8.0                           "docker-entrypoint.s..." 16 minutes ago
Up 16 minutes (healthy)   0.0.0.0:3309->3306/tcp, [::]:3309->3306/tcp quickcommerce_mysql
smailliteddy@MacBook-Air-de-Smaili archi-docker-compose %
```

La commande `docker ps` montre que les deux conteneurs `quickcommerce_app` (application) et `quickcommerce_mysql` (base de données) sont en cours d'exécution et dans l'état `healthy`, ce qui indique que l'architecture Docker fonctionne correctement.

```
smailliteddy@MacBook-Air-de-Smaili archi-docker-compose % docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED
STATUS        PORTS                                NAMES
5bfc52260226   quickcommerce-app:1.0               "docker-entrypoint.s..." 16 minutes ago
Up 16 minutes (healthy)   0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp quickcommerce_app
25293e7ef701   mysql:8.0                           "docker-entrypoint.s..." 16 minutes ago
Up 16 minutes (healthy)   0.0.0.0:3309->3306/tcp, [::]:3309->3306/tcp quickcommerce_mysql
smailliteddy@MacBook-Air-de-Smaili archi-docker-compose %
```

La commande `curl http://localhost:3000/health` retourne un statut `OK`, ce qui confirme que l'API QuickCommerce est opérationnelle et accessible sur le port 3000.

PARTIE 2 : SIMULATION DES SPOF (1h15)

Exercice 2.1 : SPOF #1 - Application Unique (15 min)

Scénario : L'application Node.js crash

Étape 1 : Mesurer le temps de réponse normal

Measure-Command { curl http://localhost:3000/health }

Noter le temps

Temps normal :

- 88.72 ms (Martin)
- 70 ms (Teddy)
- 70 ms (Axel)

Étape 2 : Arrêter le conteneur application

Arrêter seulement l'app (pas MySQL)

`docker stop quickcommerce_app`

Heure d'arrêt :

- 17H06 (Martin)
 - 17H10 (Teddy)
 - 17H42 (Axel)
-

Étape 3 : Constater l'impact

`curl http://localhost:3000/health`

Résultat :

`curl: (7) Failed to connect to localhost port 3000 after 2063 ms:`

```
smailiteddy@MacBook-Air-de-Smaili archi-docker-compose % curl http://localhost:3000/health
curl: (7) Failed to connect to localhost port 3000 after 0 ms: Couldn't connect to server
smailiteddy@MacBook-Air-de-Smaili archi-docker-compose %
```

Couldn't connect to server

Observation : Le site est totalement inaccessible,

On ne peut plus se connecter au serveur, une erreur apparait dans powershell. (Martin)

Le site est totalement inaccessible lorsque le conteneur quickcommerce_app est arrêté. (Teddy)

Étape 4 : Chronométrer la détection

Dans un contexte réel :

- Comment détecteriez-vous cette panne ?
Via un outil de monitoring (type UptimeRobot/Datadog) effectuant des tests automatiques de santé (Health Check) toutes les minutes, ou par le signalement des utilisateurs via le support client.
- Combien de temps avant détection ?
1 à 5 minutes
- Qui détecte en premier ?

Clients ou Monitoring si ça été mis en place

Étape 5 : Redémarrer l'application

`docker start quickcommerce_app`

Attendre 10 secondes

`Start-Sleep -Seconds 10`

Tester

`curl http://localhost:3000/health`

Heure de rétablissement :

- 17h19 (Teddy)
- 17h20 (Martin)
- 17h44 (Axel)

Downtime total :

- 9 minutes (Teddy)
- 14 minutes (Martin)
- 2 minutes (Axel)

Étape 6 : Calculer la perte financière

Formule : Perte = Downtime (minutes) × CA par minute

Calcul :

CA horaire : 5,000€/h

CA par minute : 5,000€ ÷ 60 = 83€/min

Downtime mesuré : 14 minutes

Perte = 9 min × 83€/min = 747€

Perte = 14min × 83€ = 1162 €

À REMPLIR

Teddy

Métrique	Valeur
Temps redémarrage app	10 secondes
Downtime total	9 minutes
Perte financière	747€

Martin

Métrique	Valeur
Temps redémarrage app	8 secondes
Downtime total	14 minutes
Perte financière	1162 €

Axel

Métrique	Valeur
Temps redémarrage app	10 secondes
Downtime total	2 minutes
Perte financière	166 €

Exercice 2.2 : SPOF #2 - Pas de Load Balancer (10 min)

Scénario : Une seule instance, pas de basculement

Observation :

Combien d'instances de l'app ?

```
docker ps | findstr quickcommerce_app
```

Résultat : 1 seule ligne

Question : Si cette instance crash, y a-t-il un basculement automatique ?

Réponse : Non

Pourquoi ? Aucune redondance

Test : Arrêter l'app et chronométrer

```
docker stop quickcommerce_app
```

Attendre 30 secondes

```
Start-Sleep -Seconds 30
```

Tester

```
curl http://localhost:3000/health
```

Résultat : Erreur (pas de basculement)

Temps sans service : 30 secondes minimum

Perte : $30s \div 60 \times 83€ = 41.5€$ pour 30 secondes !

Redémarrer :

`docker start quickcommerce_app`

 **À REMPLIR**

Solution proposée : Pour éviter ce SPOF, il faudrait :

- Déployer plusieurs instances de l'application

- Mettre en place un Load Balancer

Exercice 2.3 : SPOF #3 - Pas d'Auto-Scaling (15 min)

Scénario : Surcharge du serveur (simulation Black Friday)

Étape 1 : Test de charge normal (100 requêtes)

Measure-Command {

1..100 | ForEach-Object {

curl http://localhost:3000/api/products | Out-Null

}

}

Temps total : 1,024 secondes

Temps moyen par requête : 10,24 ms

Étape 2 : Observer les ressources

Statistiques en temps réel

docker stats quickcommerce_app

 **Noter :**

Axel

Métrique	Valeur
CPU %	0.0%
RAM utilisée	26.85 MB
NET I/O	1.07MB / 1.43MB

Étape 3 : Test de charge élevée (1000 requêtes parallèles)

```
Measure-Command {  
    $runspaces = 1..10000 | ForEach-Object {  
        $powerShell = [PowerShell]::Create()  
        [void]$powerShell.AddScript({  
            Invoke-WebRequest -Uri "http://localhost:3000/api/products" | Out-Null  
        })  
        $powerShell.BeginInvoke()  
    }  
  
    $runspaces | ForEach-Object { $_.AsyncWaitHandle.WaitOne() }  
}
```

Temps total : 11,598 secondes

Observer pendant le test :

Dans un autre terminal
docker stats quickcommerce_app

 **À REMPLIR**

Métrique	Charge normale	Charge élevée
Temps total	1,024 s	11,598 s
CPU %	0.0%	0.0%
RAM	26.85 MB	30.92MB

Étape 4 : Analyser l'impact

Questions :

1. L'application a-t-elle tenu la charge ?
Oui, L'application a réussi à répondre aux requêtes, mais le temps total d'exécution a augmenté par rapport au test normal.
2. Y a-t-il eu des ralentissements ?
Oui, Le temps nécessaire pour traiter les requêtes sous forte charge est beaucoup plus élevé que pour 100 requêtes.
3. Y a-t-il eu des erreurs ?
Non, Aucune erreur de connexion ou timeout n'a été observée pendant le test.

Observations :

Lors du test de charge élevée, l'application continue de fonctionner mais les ressources (CPU, mémoire et réseau) sont davantage sollicitées. Le temps de traitement des requêtes augmente, ce qui montre que l'architecture avec une seule instance peut devenir un point de saturation en cas de forte charge.

Étape 5 : Simulation Black Friday

Données :

- Trafic normal : 50,000 visiteurs/jour
- Trafic Black Friday : 500,000 visiteurs/jour = **x10**
- Durée pic : 4h (14h-18h)

Question : Avec une seule instance, que se passerait-il ?

Réponse :

- ☒ Ralentissement

Justification : Une seule instance doit traiter toutes les requêtes, ce qui causent les ralentissements

Exercice 2.4 : SPOF #4 - Base de Données Unique (20 min)

Scénario : MySQL crash

Étape 1 : État normal

Tester l'API

curl http://localhost:3000/api/products

Résultat : JSON avec 10 produits

Étape 2 : Arrêter MySQL

docker stop quickcommerce_mysql

Heure : h__min

Étape 3 : Tester l'impact

Health check

curl http://localhost:3000/health

Résultat : {"status":"OK"}

Question : Le health check répond-il ?

Réponse : ☒ Oui (200 OK)

Pourquoi ? : Le health check vérifie uniquement si l'application Node.js fonctionne. Même si la base de données est arrêtée, l'application est toujours en cours d'exécution et peut donc répondre au health check.

Maintenant tester l'API :

curl http://localhost:3000/api/products

Résultat attendu :

```
{  
  "error": "Database Error",  
  "message": "Erreur base de données"  
}
```

Étape 4 : Analyser la différence

Situation	Health Check	API Produits
MySQL UP	200 OK	200 OK + données
MySQL DOWN	200 OK	Erreur BDD

Conclusion :

Quand MySQL est DOWN, l'application répond au health check (elle tourne toujours) mais **toutes les fonctionnalités métier** sont en erreur.

C'est **plus subtil** qu'un crash app total !

Étape 5 : Redémarrer MySQL

docker start quickcommerce_mysql

MySQL prend plus de temps à démarrer

Start-Sleep -Seconds 30

Tester

curl http://localhost:3000/api/products

Heure de rétablissement : h__min

Downtime total : 1 minute

Perte : 1 min × 83€/min = 83 €

 **Temps de redémarrage MySQL : 30 secondes**

Question : Pourquoi MySQL prend plus de temps qu'une app Node.js ?

Réponse : MySQL doit charger la base de données, vérifier l'intégrité des fichiers et initialiser les connexions avant d'être opérationnel, ce qui prend plus de temps qu'une application Node.js qui démarre simplement son serveur.

Exercice 2.5 : SPOF #5 - Pas de Backup (15 min)

Scénario : Perte de données

Étape 1 : Sauvegarder les données actuelles

Backup de la base

```
docker exec quickcommerce_mysql mysqldump -u root -proot quickcommerce > backup_avant_test.sql
```

Vérifier la taille du fichier

```
ls -lh backup_avant_test.sql
```

Taille du backup : 5 KB

Étape 2 : Compter les produits

```
docker exec -it quickcommerce_mysql mysql -u root -proot -e "USE quickcommerce; SELECT COUNT(*) as total FROM products;"
```

Résultat attendu : 10

Nombre de produits : 10

Étape 3 : SIMULATION - Perte de données accidentelle

Supprimer tous les produits (ATTENTION : destructif)

```
docker exec -it quickcommerce_mysql mysql -u root -proot -e "USE quickcommerce; DELETE FROM products;"
```

Vérifier

```
curl http://localhost:3000/api/products
```

Résultat : [] (liste vide)

Heure de la perte : 12h20min

Étape 4 : Constater l'impact

API fonctionne mais...

```
curl http://localhost:3000/api/products
```

Résultat : [] (aucun produit)

Impact métier :

- Site accessible : ☒ Oui
 - Produits affichés : ☒ Non
 - Impact client : **Impossible d'acheter**
-

Étape 5 : Restaurer depuis le backup

Restaurer

```
docker exec -i quickcommerce_mysql mysql -u root -proot quickcommerce < backup_avant_test.sql
```

Vérifier

```
curl http://localhost:3000/api/products
```

Résultat : 10 produits de nouveau

Heure de restauration : 12h22min

Temps de perte de données : 2 minutes

Étape 6 : Questions critiques

Q1 : Si pas de backup, que se passerait-il ?

R : Toutes les données seraient perdues définitivement et il serait impossible de restaurer les produits de la base de données.

Q2 : Où est stocké le backup actuel ?

R : Sur la même machine (fichier local)

Q3 : Si la machine brûle (incendie datacenter), perd-on le backup ?

R : ☒ Oui

Q4 : Quelle est la solution ?

R : Mettre en place des sauvegardes automatiques hors site (cloud ou serveur distant) et une réplication régulière de la base de données.

À REMPLIR

Métrique	Valeur
Taille backup	5KB
Temps restauration	10secondes

Métrique	Valeur
Impact perte données	produits supprimés du catalogue
Solution proposée	backup automatique hors site

Exercice 2.6 : SPOF Bonus - Prestataire Lent (10 min)

Scénario : Le prestataire met du temps à intervenir

Simulation : Crash MySQL + Délai d'intervention

Étape 1 : Arrêter MySQL

```
docker stop quickcommerce_mysql
```

Heure de panne : 12h30

Étape 2 : Attendre 5 minutes (simulation appel prestataire)

Scénario réaliste :

1. **T+0** : MySQL crash
2. **T+2min** : Client appelle "le site ne marche pas"
3. **T+5min** : Vous contactez le prestataire
4. **T+10min** : Prestataire commence à diagnostiquer
5. **T+15min** : Prestataire redémarre MySQL

Attendre 5 minutes (ou 2 min pour le TP)

Étape 3 : Redémarrer après le délai

```
Start-Sleep -Seconds 120 # 2 minutes
```

```
docker start quickcommerce_mysql
```

Start-Sleep -Seconds 30

curl http://localhost:3000/api/products

Heure de rétablissement : 12h40min

Downtime total : 10 minutes

Perte : 10min × 83€/min = 830€

Question : Quelle serait la solution pour réduire ce délai ?

Réponses possibles :

- ☒ Monitoring automatique avec alertes
- ☒ Auto-healing (redémarrage automatique)
- ☒ Équipe interne plutôt que prestataire externe

PARTIE 3 : SYNTHÈSE (30 min)

Exercice : Tableau Récapitulatif des SPOF

À compléter avec vos mesures :

SPOF	Description	Downtime Mesuré	Perte €	Facilité Détection
#1	Application unique	1min	83€	Facile
#2	Pas de Load Balancer	1/2min	41€	Facile
#3	Pas d'Auto-Scaling	N/A (ralentissement)	Estimé: 200€	Difficile
#4	BDD unique	1min	83€	Difficile

SPOF	Description	Downtime Mesuré	Perte €	Facilité Détection
#5	Pas de backup off-site	2min	Risque: 1M€	Difficile
Bonus	Prestataire lent	10min	830€	Difficile

LIVRABLES

À RENDRE (Document PDF)

1. ☒ **Tableau récapitulatif des SPOF** (Exercice 3) avec vos mesures
2. ☒ **Captures d'écran** :
 - Docker Desktop running
 - `docker ps` montrant les 2 conteneurs
 - Test API (`curl http://localhost:3000/api/products`)
 - Erreur lors d'un crash simulé

Format : 1 document par groupe (binôme ou trinôme)

NETTOYAGE FINAL

À la fin du TP :

Arrêter les conteneurs

`docker-compose down`

Vérifier

`docker ps -a`

Supprimer les volumes (si demandé)

`docker-compose down -v`

COMMANDES DE RÉFÉRENCE

Démarrer

`docker-compose up -d`

Arrêter

`docker-compose stop`

Redémarrer

`docker-compose restart`

Arrêter et supprimer

`docker-compose down`

Voir les logs

`docker-compose logs -f`

Voir les stats

`docker stats`

Arrêter un conteneur spécifique

`docker stop quickcommerce_app`

`docker stop quickcommerce_mysql`

Redémarrer un conteneur

`docker start quickcommerce_app`

`docker start quickcommerce_mysql`

Backup BDD

`docker exec quickcommerce_mysql mysqldump -u root -proot quickcommerce > backup.sql`

Restore BDD

`docker exec -i quickcommerce_mysql mysql -u root -proot quickcommerce < backup.sql`

TROUBLESHOOTING

Problème : Docker ne démarre pas

Solution :

- Redémarrer le PC
- Vérifier que la virtualisation est activée dans le BIOS

Problème : "Port 3000 already in use"

Solution :

Trouver qui utilise le port

```
netstat -ano | findstr :3000
```

Tuer le processus

```
taskkill /PID <PID> /F
```

Problème : MySQL ne démarre pas

Solution :

Voir les logs

```
docker logs quickcommerce_mysql
```

Attendre 30-60 secondes (MySQL prend du temps)

Besoin d'aide ?

Levez la main et appelez l'enseignant ! 🙋

Bon TP ! 🚀

Vous allez **vraiment comprendre** les SPOF en les cassant vous-mêmes !